



Checklist Mini Cloud

Math-Information-Technology (Trường Đại học Đồng Tháp)



Scan to open on Studeersnel

Checklist Triển Khai Dự Án MiniCloud - Nhóm 3 Người

Tài liệu này mô tả checklist triển khai, phân công công việc và hướng dẫn kiểm thử để đạt điểm tối đa cho đồ án MiniCloud. Chia làm 3 phần chính: (I) checklist cơ bản 5 điểm, (II) checklist mở rộng cho mỗi thành viên để lấy thêm điểm, (III) hướng dẫn viết chương Demo & Kiểm thử trong báo cáo cuối kỳ.

I. CHECKLIST LÀM 5 ĐIỂM CƠ BẢN

1. Bước 1. Chạy toàn bộ hệ thống

```
docker compose build --no-cache
docker compose up -d
docker compose ps
```

2. Bước 2. Kiểm tra từng service và CHỤP MÀN HÌNH (rất quan trọng cho báo cáo)

1. Web

- Truy cập: `http://localhost:8080`

- Chụp trang Home và `/blog/`

- Lệnh test HTTP:

```
curl -I http://localhost:8080/
curl -I http://localhost:8080/blog/
```

2. Backend Flask

- Test trực tiếp:

```
curl http://localhost:8085/hello
```

- Test qua reverse proxy:

```
curl http://localhost/api/hello
```

- Kỳ vọng trả về JSON: `{"message":"Hello from App Server!"}`

3. Database (MariaDB)

- Kiểm tra schema và dữ liệu init:

```
docker exec -it db \
mysql -uroot -proot -e "USE minicloud; SHOW TABLES; SELECT * FROM notes;"
```

- Chụp terminal hiển thị bằng notes với dòng 'Hello from MariaDB!'.

4. Keycloak (Auth)

- Mở `http://localhost:8081`

- Đăng nhập: admin / admin
- Chụp màn hình dashboard Keycloak sau login.

5. MinIO

- Mở `http://localhost:9001`
- Login: minioadmin / minioadmin
- Chụp màn hình console MinIO.

6. DNS

- Test phân giải:
- ```
dig @127.0.0.1 -p 1053 web-server.cloud.local +short
```
- Chụp màn hình terminal hiển thị IP.

## 7. Prometheus

- Mở `http://localhost:9090`
- Vào Status → Targets
- Chụp màn hình target node-exporter trạng thái UP.

## 8. Grafana

- Mở `http://localhost:3000`
- Đăng nhập (thường admin/admin lần đầu)
- Chụp màn hình sau login.

## 9. Reverse Proxy

- Kiểm tra 3 route:
- ```
curl -I http://localhost/
curl -s http://localhost/api/hello
curl -I http://localhost/auth/
```
- Chụp lại output (đặc biệt /auth/ sẽ trả redirect tới Keycloak).

⇒ Chỉ cần làm xong tới đây, chụp đủ hình – nhóm đã có đủ tài liệu chứng minh phần cơ bản 5 điểm.

II. CHECKLIST MỞ RỘNG – THÊM TỪNG 0.5 ĐIỂM

Mỗi mục mở rộng ~0.5 điểm.

Người 1: Web + Proxy + Grafana (1, 8, 9, 10)

1. 1) Web blog cá nhân (0.5đ)

- Thư mục: web/html/blog/
 - Tạo thêm 3 file: blog1.html, blog2.html, blog3.html
 - Nội dung mẫu mỗi bài:

```
<h1>Kinh nghiệm học Cloud của tụi mình</h1>

<a href="/">Về trang chính</a>
```
 - Sửa web/html/index.html để có danh sách link tới 3 bài blog.
- Test:
- ```
curl -I http://localhost:8080/blog/blog1.html
```

## 2. 2) Grafana dashboard cá nhân (0.5đ)

- Truy cập http://localhost:3000
- Login admin/admin
- Add data source: Prometheus với URL http://prometheus:9090
- Tạo dashboard mới tên: System Health of <MSSV>
  - Panel CPU: node\_cpu\_seconds\_total
  - Panel Memory: node\_memory\_MemAvailable\_bytes
  - Panel Network RX: node\_network\_receive\_bytes\_total
- Chụp màn hình dashboard có 3 panel.

## 3. 3) Route /student/ trong proxy (0.5đ)

- Mở proxy/nginx.conf, thêm:  

```
location /student/ {
 proxy_pass http://app-server:8081/student;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
}
```
- Restart proxy:  

```
docker restart proxy
```
- Test:  

```
curl http://localhost/student/
```
- Chụp JSON trả về.

## 4. 4) Load Balancing round-robin (bonus 0.5đ)

- Nhân đôi web server thành web1 và web2 trong docker-compose.
- Trong proxy/nginx.conf thêm upstream:  

```
upstream web_cluster {
 server web1:80;
 server web2:80;
}
```

```
location / {
 proxy_pass http://web_cluster;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
}
```

- Sửa web/html/index.html trong từng container để in khác nhau, ví dụ 'Hello from web1' và 'Hello from web2'.

- Gọi curl http://localhost/ nhiều lần và chụp kết quả luân phiên.

## Người 2: Backend + DB + Auth (2, 3, 4)

### 1. 1) API /student (0.5đ)

- Trong thư mục app/, tạo students.json:

```
[
 {"id":1,"name":"Nguyen A","major":"Cloud","gpa":3.2},
 {"id":2,"name":"Tran B","major":"DevOps","gpa":3.5},
 {"id":3,"name":"Le C","major":"Security","gpa":3.8}
]
```

- Trong app/app.py thêm route:

```
import json
@app.get("/student")
def student():
 with open("students.json") as f:
 data = json.load(f)
 return jsonify(data)
```

- Rebuild app-server:

```
docker compose build app-server --no-cache
```

```
docker compose up -d app-server
```

- Test:

```
curl http://localhost/api/student
```

- Chụp JSON trả về.

### 2. 2) Database studentdb (0.5đ)

- Tạo file db/init/002\_student.sql:

```
CREATE DATABASE IF NOT EXISTS studentdb;
USE studentdb;
CREATE TABLE IF NOT EXISTS students (
 id INT AUTO_INCREMENT PRIMARY KEY,
 student_id VARCHAR(10),
 fullname VARCHAR(100),
 dob DATE,
```

```
major VARCHAR(50)
);
INSERT INTO students (student_id, fullname, dob, major) VALUES
('SV001','Nguyen Van A','2003-05-12','Cloud'),
('SV002','Tran Thi B','2004-09-21','DevOps'),
('SV003','Le Van C','2003-12-02','Security');
```

- Reset DB container để nó chạy init mới:  
docker compose down db  
docker compose up -d db
- Kiểm tra:  
docker exec -it db \
- mysql -uroot -p -e "USE studentdb; SELECT \* FROM students;"
- Chụp màn hình bảng students.

### 3. 3) Keycloak realm riêng + test /secure (0.5đ)

- Vào http://localhost:8081 → login admin/admin
- Tạo Realm mới (ví dụ: realm\_sv001)
- Trong realm\_sv001:
  - Tạo 2 user: sv01, sv02 (set password)
  - Tạo 1 client tên flask-app (public)
- Lấy access token của user sv01 (qua Keycloak token endpoint)
- Gọi API /secure:  
curl -H "Authorization: Bearer <ACCESS\_TOKEN>" http://localhost/api/secure
- Chụp màn hình trả về JSON kiểu:  
{"message":"Secure resource OK","preferred\_username":"sv01"}

### Người 3: MinIO + DNS + Prometheus (5, 6, 7)

#### 1. 1) MinIO upload avatar & tài liệu (0.5đ)

- Mở http://localhost:9001 → login minioadmin/minioadmin
- Tạo bucket profile-pics → upload avatar.jpg
- Tạo bucket documents → upload file PDF báo cáo nhóm
- Chụp màn hình giao diện MinIO hiển thị bucket và file.

#### 2. 2) DNS record mở rộng (0.5đ)

- Mở dns/db.cloud.local và thêm:  
app-backend IN A 10.10.10.20  
minio IN A 10.10.10.30  
keycloak IN A 10.10.10.40
- Restart DNS:  
docker restart dns-server
- Kiểm tra bằng dig:

```
dig @127.0.0.1 -p 1053 app-backend.cloud.local +short
dig @127.0.0.1 -p 1053 minio.cloud.local +short
dig @127.0.0.1 -p 1053 keycloak.cloud.local +short
```

- Chụp terminal với IP trả về.

### 3. 3) Prometheus scrape web (0.5đ)

- Mở monitor/prometheus.yml
- Thêm job mới:
  - job\_name: 'web'
  - static\_configs:
    - targets: ['web:80']
- Restart Prometheus:  
docker restart prometheus
- Vào <http://localhost:9090> → Status → Targets
- Chụp màn hình thấy job web trạng thái UP.

## III. CHƯƠNG 4 (DEMO & KIỂM THỬ) GỢI Ý VIẾT BÁO CÁO

Trong báo cáo cuối kỳ, Chương 4 có thể theo format lặp lại cho từng server như sau:

1. Mục tiêu kiểm thử: mô tả mục tiêu của service.
2. Lệnh đã dùng: dán các lệnh curl / dig / mysql / docker exec.
3. Kết quả kỳ vọng: mô tả kết quả mong muốn (HTTP 200 OK, JSON trả về, login thành công...).
4. Ảnh minh chứng: chụp màn hình UI hoặc terminal.

Áp dụng y chang cho: Web (/ và /blog), App (/hello, /student), DB (SELECT \*), Keycloak login và /secure, MinIO console, DNS dig, Prometheus Targets, Grafana dashboard, Proxy (/api và /student).

## TÓM LẠI

1. ZIP hiện tại = đủ 5 điểm cơ bản, chỉ cần chạy lên và chụp hình chứng minh.
2. Làm thêm các bước mở rộng ở trên = gom thêm 4.5 điểm + bonus 0.5 round-robin.
3. Báo cáo: làm theo khung Chương 1 → 5 của đề, và dùng Chương 4 như hướng dẫn ở trên.

Mỗi người chỉ cần làm đúng phần mình, chụp hình đầy đủ, rồi ghép lại vào báo cáo chung.